

DOKUMENTASI TEKNIS

A. Arsitektur Sistem

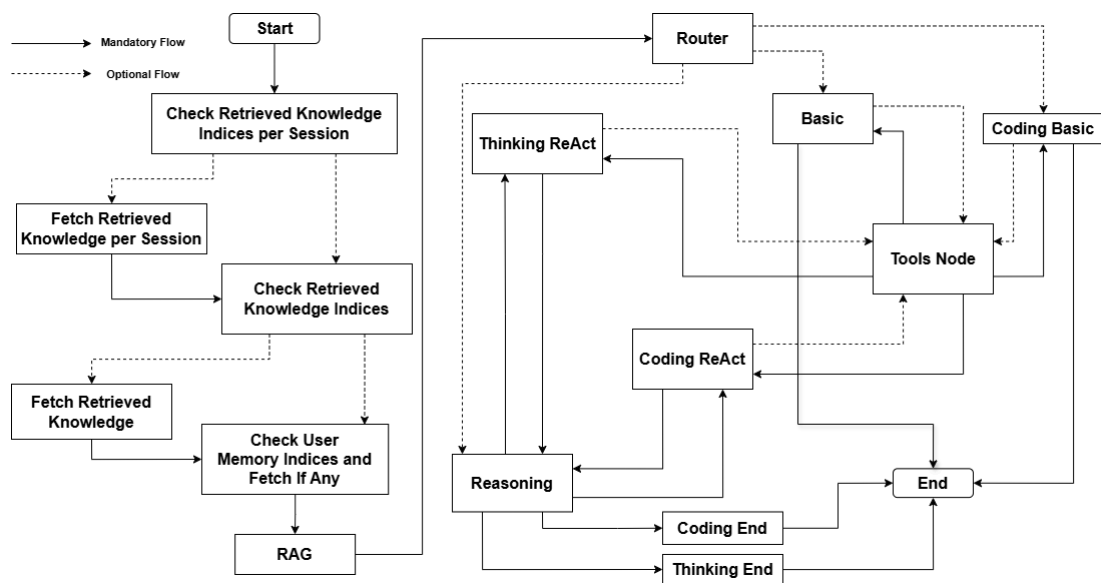
Sistem Chatbot Nusantara merupakan sistem *chatbot general* RAG yang dibuat dengan *framework* Langgraph dan Python. Langgraph adalah *framework open source* dari LangChain yang digunakan untuk membangun dan mengelola alur AI Agent menggunakan struktur berbasis graf. Hal ini memungkinkan pengembang untuk membuat *workflow* sebagai *node* dan *edge*, sehingga interaksi agen yang kompleks menjadi lebih terstruktur, terukur, dan lebih mudah dikontrol. Dalam Langgraph terdapat struktur data yang disebut dengan *state*. *State* pada Langgraph merupakan mekanisme utama untuk komunikasi antar *node*. Selain itu, *State* juga bertindak sebagai pesan terstruktur yang merangkum *snapshot* sistem saat ini pada setiap momen tertentu. Tabel 1 merupakan *State* data yang digunakan di dalam sistem Chatbot Nusantara.

Tabel 1. Struktur Data State Langgraph Sistem Chatbot Nusantara

Data	Definisi
tenant_id	ID unik setiap tenant/perusahaan
user_id	ID unik setiap pengguna
thread_id	ID unik setiap chat pengguna
mode	Mode yang digunakan oleh pengguna. Terdiri dari “auto”, “thinking”, atau “fast”
messages	Pesan dari pengguna, AI, dan Tools
selected_knowledge	Berisi indeks <i>knowledge tenant</i> yang di- <i>retrieve</i>
chunk_selected_knowledge	Berisi setiap <i>chunk</i> yang sesuai dengan <i>knowledge</i> ID pada selected_knowledge
retrieved_session_knowledge	Berisi indeks <i>knowledge</i> per <i>chat</i> yang di- <i>retrieve</i>
chunk_retrieved_session_knowledge	Berisi setiap <i>chunk</i> yang sesuai dengan <i>knowledge</i> ID pada

	retrieved_session_knowledge
memory_ids	Berisi indeks memori per <i>user</i> yang di- <i>retrieve</i>
memory	Berisi setiap <i>chunk</i> yang sesuai dengan memori ID pada memory_ids
last_query	Pertanyaan <i>query</i> untuk keperluan <i>reasoning</i> yang di- <i>generate</i> AI
iteration	Iterasi <i>reasoning</i> yang sudah dilakukan, maksimal 3 kali
route	Rute yang ditentukan oleh <i>node router</i>
reasoning_questions_observation	Hasil <i>reasoning</i> pada <i>node reasoning</i>

Komponen lain pada Langgraph adalah *checkpoint*. *Checkpoint* pada Langgraph berguna untuk mempertahankan data *state* di dalam dan di seluruh interaksi sistem graf. *Checkpoint* juga merupakan cuplikan *state* graf pada titik waktu tertentu yang diidentifikasi oleh ID unik, yang meningkat secara monoton. Gambar 1 menunjukkan diagram alir sistem Chatbot Nusantara yang memiliki kondisional *knowledge* dan berbagai macam *router*.



Gambar 1. Flowchart Sistem Chatbot Nusantara

Berikut merupakan komponen diagram alir Chatbot Nusantara yang sesuai dengan Gambar 1.

1. Check Retrieved Knowledge Indices per Session

Node ini untuk mengecek, apakah pada *checkpoint/cycle* sebelumnya terdapat *knowledge* per sesi (*chat*) yang disimpan.

2. Fetch Retrieved Knowledge per Session

Jika saat pengecekan terdapat data indeks *knowledge* per sesi yang disimpan, maka ambil *chunk knowledge* per sesi tersebut berdasarkan indeksnya.

3. Check Retrieved Knowledge Indices

Node ini untuk mengecek, apakah pada *checkpoint/cycle* sebelumnya terdapat *knowledge tenant* yang disimpan.

4. Fetch Retrieved Knowledge

Jika saat pengecekan terdapat data indeks *knowledge* per *tenant* yang disimpan, maka ambil *chunk knowledge* per *tenant* tersebut berdasarkan indeksnya.

5. Check User Memory Indices and Fetch If Any

Node ini untuk mengecek, apakah pada *checkpoint/cycle* sebelumnya terdapat memori pengguna yang disimpan. Jika saat pengecekan terdapat data indeks memori pengguna yang disimpan, maka ambil *chunk* memori pengguna tersebut berdasarkan indeksnya.

6. RAG

Node ini untuk mengambil data *chunk knowledge* per *tenant* yang sudah diurutkan berdasarkan skor *cosine similarity* yang paling besar. Jika terdapat *knowledge* per *tenant* pada *database*, maka ambil lima data *chunk* dengan skor terbesar.

7. Router

Node ini sebagai pengarah untuk menentukan *agent* mana yang diperlukan untuk memproses *prompt* dari pengguna. Terdapat tiga pilihan, yaitu *Basic*, *Coding Basic*, dan *Reasoning* (*Coding ReAct* atau *Thinking ReAct*).

8. Basic Agent

Agent ini memproses *prompt* dari pengguna pada tugas yang sederhana, seperti menjawab obrolan sederhana, menerjemahkan kalimat, meringkas suatu teks, atau tugas penulisan sederhana.

9. Coding Basic

Agent ini memproses *prompt* dari pengguna pada tugas *coding* sederhana, seperti penulisan sintaks, penulisan kode sederhana, penjelasan kode, memperbaiki *bug* kecil, atau membuat kode ringan.

10. Reasoning Agent

a. Coding ReAct

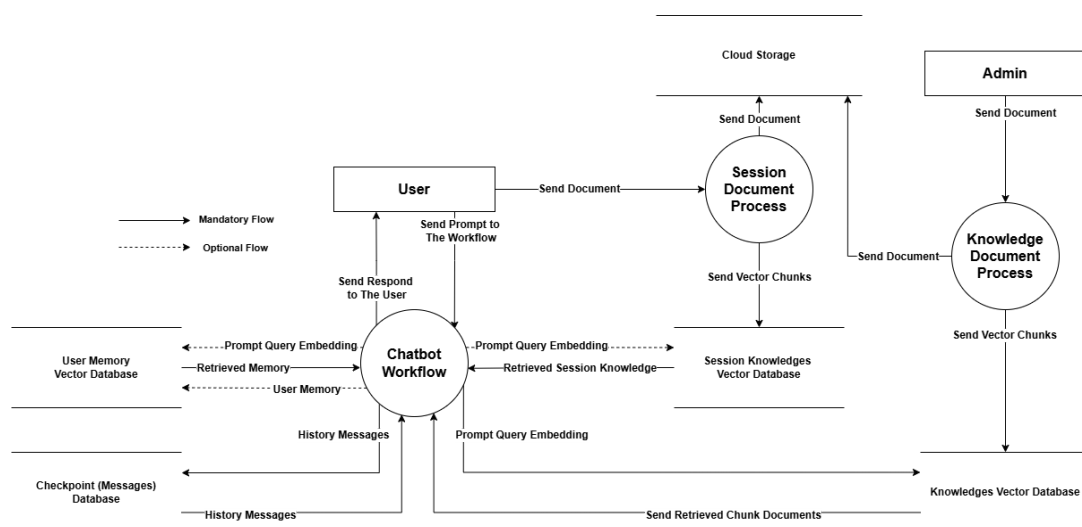
Agent ini merupakan *agent coding* yang memerlukan analisis mendalam, membuat desain arsitektur, *debugging* error yang kompleks, membangun sistem, atau optimasi performa.

b. Thinking ReAct

Agent ini merupakan *agent thinking* yang memerlukan pemikiran mendalam, seperti menulis essay, melakukan evaluasi, perencanaan, pembuat keputusan, atau *problem solving* yang kompleks.

B. Diagram Alir Data

Sistem ini memiliki diagram alir data yang dapat ditunjukkan pada Gambar 2. Diagram ini terdiri dari beberapa *database*, yaitu *database* untuk menyimpan memori pengguna (User Memory), menyimpan riwayat percakapan (Checkpoint/Messages), menyimpan potongan dokumen yang dikirim oleh pengguna (Session Knowledges) ataupun admin (Knowledges), serta *storage cloud* untuk menyimpan dokumen hasil unggahan pengguna/admin. Semua database ini disimpan pada *database* dan *storage cloud* Supabase.

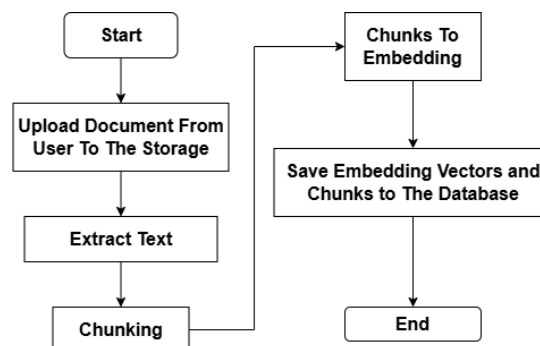


Gambar 2. Diagram Alir Data Chatbot Nusantara

Berdasarkan Gambar 2, apabila pengguna mengirim *prompt* ke *chatbot*, maka *prompt* tersebut diterima dan diproses langsung oleh *workflow*. *Workflow* memproses *prompt* tersebut apakah memerlukan data dari *database* atau tidak, seperti memori pengguna, *knowledge* per sesi, atau dari *database knowledge tenant*. *Workflow* selalu menyimpan riwayat *checkpoint*/percakapan ke *database*. Apabila pengguna mengirim dokumen, maka dokumen tersebut diproses dan disimpan langsung ke *storage cloud*, serta potongan isi dokumen tersebut disimpan ke *vector database cloud*. Apabila admin mengunggah dokumen, dokumen tersebut akan diproses, kemudian dikirim ke *storage cloud*, serta mengirim potongan isi dokumennya ke *vector database cloud*.

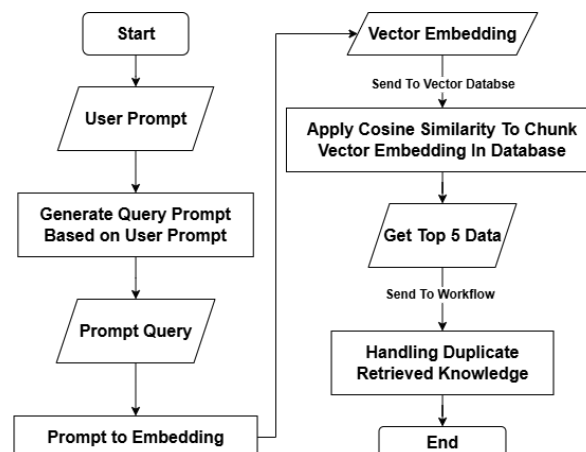
C. RAG Pipeline

Sistem Chatbot ini menggunakan RAG sebagai *knowledge* tambahan pada LLM. Proses RAG ini memerlukan dokumen dari pengguna yang akan diproses menjadi *knowledge*, yang dapat digunakan sebagai pengetahuan tambahan LLM. Proses RAG ini dapat dilihat pada Gambar 3 dan Gambar 4. Pada Gambar 3, pengguna atau admin meng-*upload* dokumen pdf atau txt ke dalam *workflow*. Kemudian, secara otomatis, dokumen tersebut disimpan ke *cloud storage* Supabase. Setelah itu, ekstrak teks dokumen, lalu potong (*chunking*) setiap isi dokumen tersebut setiap 500 karakter, dengan *overlap* 50 karakter di awal dan akhir *chunk*. Strategi ini dipilih karena setiap potongan informasi agar tetap memiliki konteks yang cukup untuk dipahami, sekaligus menghindari hilangnya konteks/informasi di batas awal dan akhir antar *chunk* akibat pemotongan yang terlalu ketat. Lalu, setiap *chunk* diubah ke dalam *vector embedding*, kemudian diunggah ke *database* Supabase beserta isi *chunk*-nya.



Gambar 3. Proses Ekstrak Dokumen Hingga Disimpan ke Vector Database

Pada Gambar 4, setiap *prompt* user yang dikirim, diproses terlebih dahulu di *node* RAG. Proses ini diawali dengan *generate query* dalam bentuk pertanyaan berdasarkan *prompt* dari pengguna. Setelah dihasilkan *query* dalam bentuk pertanyaan, *query* ini diubah ke dalam bentuk *vector embedding*. Data *vector* ini kemudian dikirim ke *database*, lalu hasilnya dikirim ke *workflow* dengan urutan sesuai skor *cosine similarity* antara *vector query* dengan masing-masing *vector chunk*. *Cosine similarity* dipilih karena metrik ini mengukur kemiripan arah “makna/konteks” *vector*, bukan jarak *vector* absolut, sehingga tidak terpengaruh oleh panjang-pendek potongan teks. Data yang diambil dibatasi hingga 5 data teratas dengan skor tertinggi. Lima data ini dipilih karena agar dapat menjaga *context window* LLM yang digunakan, sehingga proses *prompt* oleh LLM bisa menjadi lebih efisien dan tidak melebihi batas token, serta untuk mencegah *noise* (kehilangan konteks) pada data *chunk*. Setelah data terambil dari *database*, data *chunk* ini kemudian di filter agar tidak duplikat dengan data *knowledge* yang sebelumnya pernah diambil.



Gambar 4. Proses Retrieval pada RAG

D. Model & Teknologi (*Tech Stack*)

1. Model

Terdapat beberapa model yang digunakan pada sistem *chatbot* ini. Model yang digunakan oleh sistem ini adalah Gemini 3.1 Flash Lite dan Gemini Embedding 1. Model ini dipilih karena selain API gratis dengan batasan, model ini mudah untuk diintegrasikan tanpa infrastruktur tambahan yang berat dan kompleks.

2. Teknologi

Sistem chatbot ini menggunakan beberapa teknologi, yaitu Langgraph, FastAPI, dan Python. *Framework* Langgraph ini dipilih karena lebih bebas dan fleksibel untuk digunakan. Selain itu, cocok untuk membuat *agentic workflow* yang kompleks, seperti *multi-step reasoning* atau *conditional branching* antar proses. FastAPI digunakan untuk men-*deploy agent* Langgraph yang sudah jadi agar bisa digunakan oleh *client*. FastAPI digunakan karena mendukung *async/await* secara *native*, lebih ringan, dan menghasilkan dokumentasi API yang interaktif (Swagger). Semua sistem ini menggunakan bahasa pemrograman Python (3.10).

3. Database

Semua data hasil *generate AI*, informasi pengguna, dan dokumen pengguna, disimpan pada *cloud database* Supabase. *Database* ini dipilih karena mudah diintegrasikan, tidak membutuhkan infrastruktur tambahan, serta menyediakan PostgreSQL sebagai basis data, sehingga mendukung *query* relasional dan ekstensi pgvector untuk kebutuhan *vector search*.

4. Model Routing

Sistem *chatbot* ini menggunakan model *routing* untuk mengarahkan *agent* mana yang digunakan. Pada sistem ini terdapat 4 *agent*, yaitu *agent basic*, *coding basic*, *coding react*, dan *thinking react*. *Prompt* yang dimasukkan pada model *router* sudah diformat sedemikian sehingga *output* formatnya selalu antara 4 rute, yaitu “basic”, “coding_basic”, “coding_react”, dan “thinking_react”. Model *router* dibagi menjadi empat karena setiap *agent* dioptimalkan untuk tugas yang berbeda-beda, sehingga respons yang dihasilkan lebih akurat dan relevan.

5. Prompt Management

Prompt dan *query* data di sistem ini disimpan pada file yang berbeda, sehingga pengelolaannya lebih mudah dan terstruktur. Jadi, ketika terdapat *node* yang membutuhkan *prompt* atau *query*, module ini bisa langsung digunakan di dalam *script* tersebut tanpa memakan banyak baris.

E. Cara Menjalankan

Dalam praktik kali ini, penulis menggunakan Swagger dari FastAPI untuk melakukan demonstrasi sistem *chatbot*. Terdapat tiga *endpoint* utama yang dapat dijalankan secara langsung, yang terdiri dari `/upload`, `/chat`, dan `/health`, serta dengan tambahan *endpoint* `/add_member`.

1. Inisialisasi Sistem

- a. Git clone repository berikut [chatbot-nusantara](https://github.com/fardhan248/chatbot-nusantara)

```
git clone
https://github.com/fardhan248/chatbot-nusantara.git
```

- b. Masuk ke direktori `chatbot-nusantara`

```
cd chatbot-nusantara
```

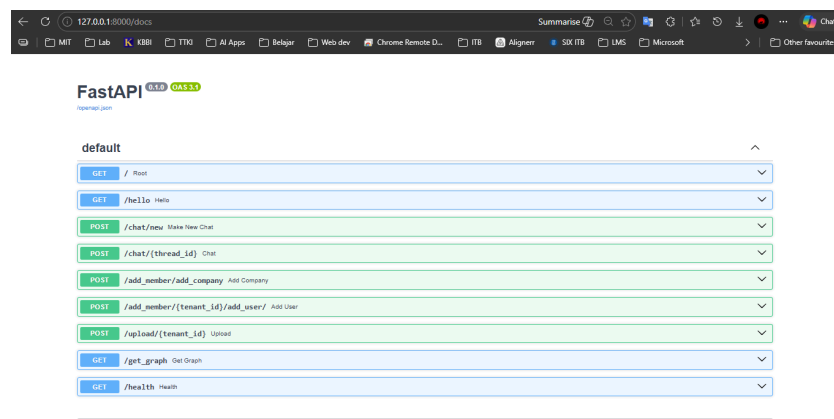
- c. Install requirement dengan perintah berikut

```
pip install -r requirements.txt
```

- d. Jalankan FastAPI

```
fastapi dev main_app.py
```

- e. Salin link `http://127.0.0.1:8000/docs` ke browser



2. Add Member

- Untuk menambahkan nama *tenant*, gunakan *endpoint* `/add_member/add_company`
- Masukkan nama *tenant*, lalu klik “Execute”

POST /add_member/add_company Add Company

Parameters

No parameters

Request body *required* application/json

Edit Value | Schema

`{
 "company_a":
}`

Execute

c. Contoh respons sukses

Responses

Curl

```
curl -X POST \
  http://127.0.0.1:8080/add_member/add_company \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{  
  "company_a":  
}'
```

Request URL

http://127.0.0.1:8080/add_member/add_company

Server response

Code Details

200

Response body

```
{  
  "status": "success",  
  "values": {  
    "tenant_id": "e5f6d7fe-6b72-4f15-8bf-e8ed96a7d295",  
    "name": "company_a"  
  }  
}
```

Response headers

```
access-control-allow-credentials: true  
access-control-allow-origin: *  
content-length: 188  
content-type: application/json  
date: Tue, 12 May 2026 07:50:40 GMT  
server: ocscore
```

d. Tampilan di *database*

	tenant_id	name	created_at
	e5f6d7fe-6b72-4f15-8bf-e8ed96a7d295	company_a	2026-05-12 07:50:39.336909+00

- Untuk menambahkan *user*, dapat menggunakan *endpoint* `/add_member/{tenant_id}/add_user`
- Masukkan *tenant_id* pada parameter URL, nama *user*, dan *role*-nya, lalu klik “Execute”

POST /add_member/{tenant_id}/add_user Add User

Parameters

Name Description

tenant_id *required* e5f6d7fe-6b72-4f15-8bf-e8ed96a7d295
string(uuid)
(path)

Request body *required* application/json

Edit Value | Schema

```
{  
  "user": "user_a",  
  "role": "user"  
}
```

Execute

g. Contoh respons sukses

The screenshot shows a REST client interface with the following details:

- Request URL:** `http://127.0.0.1:8080/add_user/e5f6d7fe-6b72-4f15-8fbf-e8ed96a7d295/add_user`
- Request Body (JSON):**

```
{  "tenant_id": "e5f6d7fe-6b72-4f15-8fbf-e8ed96a7d295",  "role": "user"}
```
- Response Code:** 200
- Response Body (JSON):**

```
{  "status": "success",  "values": {    "user_id": "920881e0-862-4bae-bc7-5768b71a8a",    "tenant_id": "e5f6d7fe-6b72-4f15-8fbf-e8ed96a7d295",    "user": "user_2",    "role": "user"  }}
```
- Response Headers:**

```
access-control-allow-credentials: true  access-control-allow-origin: *  content-length: 361  content-type: application/json  date: Tue, 12 May 2020 08:01:55 GMT  server: uvicorn
```

h. Tampilan di database

		tenant_id	uuid	role	text	created_at	timestampz	name	bytes (hex)
		e-bcc7-57608e73a0b1	e5f6d7fe-6b72-4f15-8fbf-e8ed96a...	user		2026-05-12 08:01:54.52809+00		1x28f4c6aef60a34925bd4b	

3. Upload

- Gunakan *endpoint* /upload
- Masukkan *tenant_id* dan *upload* file, lalu klik “Execute”

The screenshot shows a REST client interface for the `/upload/{tenant_id}` endpoint. The **Parameters** tab is active, showing the `tenant_id` parameter with the value `f9c6051e-09b9-40db-9cb4-ebdd605015ba`. The **Request body** is set to `multipart/form-data`. A file named `Portfolio.pdf` is selected for upload. The **Execute** button is visible at the bottom.

c. Contoh respons sukses

The screenshot shows the **Response body** of a successful upload:

```
{  "status": "success",  "knowledge_id": "415fb2cf-1a6b-4422-8af3-cb8501713665",  "tenant_id": "f9c6051e-09b9-40db-9cb4-ebdd605015ba",  "me": {    "adadata": {      "filename": "Portfolio.pdf",      "content-type": "application/pdf",      "pages": 3,      "len_chunks": 7    },    "chunk_ids": [      "07b2f736-7ada-4507-a253-719ddbfb5c1b",      "da7eebcb-ae3-4591-b204-f51ad82ec2e4",      "5da5ff18-84e2-41de-a588-7ad819ad3b41",      "709567b3-9ef4-48a2-b514-50a164d8bd60",      "c58ded61-299a-4a45-b02d-1f4261473201",      "31ade4c7-85e1-481d-8b63-5e77aff2e797",      "d7d895b7-17b3-47e5-917d-5b02c347a64d"    ]    }  }
```

d. Tampilan di *database*

chunk_id	uid	knowledge_id	uid	tenant_id	uid	content	bytes (hex)
07b2f736-7ada-4507-a253-79d9dbfb5c1b		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x0a6e6d6cddfd4579ba78b9bd3de06i	
31ade4c7-85e1-481d-b653-5e77eff2e797		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x784076e8c0bec86e4825373458c8d:	
5da5ff18-84e2-41de-a588-7ad919ad3b41		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x465b5d713b3f4a4664f27dc27c89e7	
709567b3-9ef4-48a2-b514-50a164d8bd61		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x094118b946be5d9909bfbee06b6a2i	
c58ded61-299a-4a45-b02d-1f4261473201		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x6f5af02ad985a69c0e3c83c5dea12c:	
d7d895b7-17b3-47e5-917d-5b02c347a64d		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x4c91d09625726445fa62c38131848f	
da7eebcb-ae3-4591-b204-f51ad82ec2e4		415fb2cf-1a6b-4422-8af3-cb8507...		f9c6051e-09b9-40db-9cb4-ebdd...		\x836e67e637b9169f0747e7f553679e7	

4. Chat

- Untuk *chat* baru, gunakan *endpoint* /chat/new
- Masukkan *prompt* dan file dokumen (opsional), lalu klik “Execute”

POST /chat/new Make New Chat

Parameters

No parameters

Request body *required* multipart-form-data

input_data *required* string

tenant_id: 15cd051e-09b9-40db-9cb4-ebdd00015ea, \

Choose File White Green and Blue ...imple Presentation.pdf

string | (string | null)(binary)

Send empty value

Execute

c. Contoh respons sukses

```
{
  "thread_id": "0172c2a0-02f7-4e43-84c6-0b5a6992454f",
  "content": "Halo! Kabar saya baik, terima kasih sudah bertanya. Semoga Anda juga dalam keadaan sehat.\n\nBerikut adalah ringkasan dari dokumen yang Anda unggah mengenai 'Chatbot Nusantara', sebuah sistem AI Agent berbasis RAG yang dikembangkan menggunakan 'LangGraph' dan 'Python'. **Tujuan:** Membuat chatbot general RAG yang mampu menjawab pertanyaan pengguna untuk memberikan jawaban yang spesifik.\n\n**Arsitektur Sistem:**\n- Menggunakan struktur berbasis graf (LangGraph) untuk mengelola alur kerja AI Agent.\n- Terdiri dari beberapa komponen utama: **State** untuk komunikasi antar-node dan **Checkpoint** untuk menyimpan snapshot sistem.\n- Data state mencakup informasi seperti tenant_id, user_id, thread_id, mode (auto, thinking, final), serta **messages** dan data terkait retrieval (knowledge/memory).\n- **RAG** (Retrieval-Augmented Generation) untuk menghasilkan jawaban.\n- **Model** (misalnya LLaMA, GPT-4o, Gemini, dll) untuk menghasilkan jawaban.\n- **Database** untuk menyimpan informasi.\n- **Fitur & Model Routing:** Mendukung beberapa model routing seperti 'basic', 'coding', 'coding react', dan 'thinking react'. **Status Pengembangan:**\n- Saat ini dalam kondisi fitur 'streaming', beberapa fitur belum berfungsi, dan proses upload dokumen masih dilakukan satu per satu.\n- Rencana Pengembangan: Implementasi chat 'streaming', penambahan 'node judge' (LLM kemampuan upload multi-dokumen, integrasi LLM khusus 'coding', dan pengembangan 'agent orchestration' yang lebih kompleks).\n\nSecara keseluruhan, Chatbot Nusantara adalah proyek sistem AI terstruktur yang dirancang untuk mendukung alur kerja yang fleksibel dan informatif melalui integrasi dengan berbagai komponen. Apakah ada bagian spesifik dari arsitekturnya yang ingin Anda diskusikan lebih lanjut?"
```

d. Tampilan di *database*

thread_id	uid	tenant_id	uid	user_id	uid	created_at	timestamp	title
0172c2a0-02f7-4e43-84c6-0b5a6992454f		f9c6051e-09b9-40db-9cb4-ebdd...		b509ccae-a056-4962-b91b-fa071...		2026-05-12 12:58:53.675675+01		\x84a
59b0ed61-5bb3-436d-ba91-c9a44aa7ad2		f9c6051e-09b9-40db-9cb4-ebdd...		b509ccae-a056-4962-b91b-fa071...		2026-05-12 12:52:38.580981+01		\x2f5f

- e. Untuk *chat* lama atau yang sudah ada, gunakan *endpoint* `/chat/{thread_id}`

The screenshot shows a REST client interface for the `/chat/{thread_id}` endpoint. The `thread_id` path parameter is set to `59b0ed61-5bb3-436d-ba91-c9a44aa7ad28`. The request body is configured as `multipart/form-data`. It includes a required `input_data` string field with the value `{\"tenant_id\": \"9b051e-080-40b-9eb-4e0d05015a\", \"`. There is a file upload section with a `Choose File` button and a `Send empty value` checkbox. A blue `Execute` button is at the bottom.

- f. Contoh respons sukses

The screenshot shows the response body of a successful chat request. The JSON response is:

```
{  \"thread_id\": \"59b0ed61-5bb3-436d-ba91-c9a44aa7ad28\",  \"content\": \"Nama teman Anda yang sama-sama menyukai topik AI adalah **Budi**.\"}
```

- g. Contoh respons ketika bertanya tentang waktu saat ini

The screenshot shows the response body of a chat request asking for the current time. The JSON response is:

```
{  \"thread_id\": \"59b0ed61-5bb3-436d-ba91-c9a44aa7ad28\",  \"content\": \"Saat ini adalah tanggal 12 Mei 2026, pukul 20:36 WIB.\"}
```

- h. Contoh respons ketika bertanya yang membutuhkan kalkulator

The screenshot shows the response body of a chat request asking for a calculation. The JSON response is:

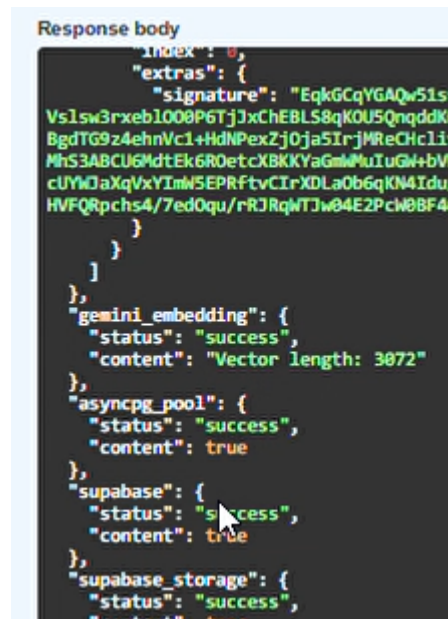
```
{  \"thread_id\": \"59b0ed61-5bb3-436d-ba91-c9a44aa7ad28\",  \"content\": \"Luas lingkaran dengan jari-jari 3 cm adalah sekitar 28,27 cm².\"}
```

5. Health

- a. Gunakan *endpoint* `/health`, lalu klik “Execute”

The screenshot shows the `/health` endpoint in a REST client. The method is `GET`. There are no parameters. A blue `Execute` button is visible. Below, the response section shows a `200 Successful Response` with a media type of `application/json`. There are tabs for `Example Value` and `Schema`.

b. Contoh respons sukses



```
Response body
{
  "index": 0,
  "extras": {
    "signature": "EqkGcQYGAQw51s9
Vslsw3rxeb1008P6Tj3xChEBLS8qKOU5QnqddKC
BgdTG9z4ehnVc1+HdNPexZj0ja5IrjWRcHcliv
MhS3ABCUGMdtEk6R0etcXBKKYaGmMuIuGW+bVf
cUYWJaXqVxYImWSEPRFtvCIrXDLa0b6qKN4Idu/
HVFQRpchs4/7ed0qu/rRJRqWTJw04E2Pch08F46
"
  }
},
  "gemini_embedding": {
    "status": "success",
    "content": "Vector length: 3072"
  },
  "asynpg_pool": {
    "status": "success",
    "content": true
  },
  "supabase": {
    "status": "success",
    "content": true
  },
  "supabase_storage": {
    "status": "success",
    "content": true
  }
}
```

F. Trade-Off Teknis

1. Karena infrastruktur yang digunakan masih kurang memadai, tidak memungkinkan untuk menjalankan LLM secara lokal. Jadi, model yang digunakan masih model API *open source*, sehingga terdapat kendala seperti batasan *request* dan token.
2. Karena menggunakan model LLM berbasis API, internet sangat berpengaruh terhadap penerimaan respons dari penyedia. Berbeda dengan model lokal yang prosesnya terjadi di mesin sendiri.
3. Data pengguna dan dokumen masih disimpan di *cloud database* eksternal, sehingga kontrol privasi data terbatas. Berbeda dengan *database* lokal yang disimpan di infrastruktur sendiri.

G. Limitasi Sistem

1. Sistem *chatbot* ini masih belum terdapat fitur *streaming chat*, sehingga pengguna harus menunggu seluruh jawaban selesai di-generate sebelum ditampilkan, yang dapat terasa lebih lambat.
2. Beberapa fungsi *tool* masih belum bisa terpanggil. Hal ini karena terdapat kendala *script* yang masih harus dievaluasi lebih lanjut. Tapi, beberapa *tool* sederhana seperti `get_datetime_now` dan `calculator` sudah berhasil dijalankan.

3. Belum adanya *node judge* LLM saat pengambilan dokumen, sehingga meningkatkan risiko terambilnya *chunk* yang tidak relevan. Akibatnya, model dapat menghasilkan jawaban yang tidak relevan dan model juga men-*generate* jawaban lebih lama karena boros token *context window*.
4. Sistem ini masih menerima hanya satu dokumen, sehingga apabila admin/*user* ingin meng-*upload* dokumen lebih dari satu, harus diunggah satu per satu.

H. Rencana Pengembangan Lanjutan

1. Menambahkan fitur *chat streaming* agar dapat meningkatkan *user experience* pada sistem *chatbot* yang dibuat.
2. Mengevaluasi lebih dalam terkait *tool* yang masih belum bisa terpanggil oleh LLM atau *workflow*.
3. Menambahkan *node judge* LLM untuk menentukan apakah *chunk* yang terambil relevan atau tidak, sehingga tidak boros token LLM.
4. Menambahkan fitur *upload* dokumen yang bisa unggah lebih dari satu dokumen.
5. Menambahkan model LLM yang sudah di-*training* (*open source* atau API) dengan data khusus *coding*.
6. Menambahkan *agent orchestration* untuk membuat laporan atau melakukan task yang lebih kompleks.
7. Menambahkan *tools* lain, seperti kondisi cuaca, *run script coding*, dll